

Software Requirements Specification (SRS) & Development Instruction

Project: Public Key Infrastructure (PKI) Issuing Certificate Authority (CA) System

1. Project Objective & Scope

1.1 Purpose

This document establishes the definitive functional, technical, security, operational, and non-functional engineering requirements for the **PKI Issuing Certificate Authority (CA) System**. The platform delivers a centralized, enterprise-grade web interface to manage the complete lifecycle of digital certificates, corporate cryptographic keys, and real-time high-availability validation services, anchored securely to Hardware Security Modules (HSMs).

1.2 System Scope

The system explicitly encompasses:

- . Issuing CA management, hierarchical profile configuration, and CP/CPS governance.
- . End-to-end Certificate lifecycle operations (Issuance, Renewal, Suspension, Revocation).
- . Registration Authority (RA) user vetting pipelines and multi-step CSR approval workflows.
- . High-availability Validation Services (OCSP responders and CRL/Delta-CRL publishing engines).
- . Hardware-isolated cryptographic security via local or remote HSM integrations (PKCS#11).
- . Role-Based Access Control (RBAC), Multi-Factor Authentication (MFA), and immutable security auditing.
- . Comprehensive reporting, automated alert notifications, and disaster recovery orchestration.

2. Technical Architecture & Tool Stack Framework

The system utilizes a systematically decoupled, multi-tier deployment architecture. The required technologies and specific infrastructure tooling are defined below:

2.1 Software and Framework Stack

- . **Web UI Layer:** React, TailwindCSS, Axios (state management, interactive components, responsive CSS frameworks).
- . **API Layer (REST Controllers):** Spring Boot Web Starter, Spring Security, Nimbus JOSE-JWT (stateless JWT token processing, JSON schema payloads).
- . **Service Layer (Business Logic):** Spring Boot Core Framework, Spring Validation, Spring Data JPA.
- . **Crypto Service Layer:** Bouncy Castle Provider (BC-FIPS or standard), SunPKCS11 cryptographic wrapper.
- . **Persistence Layer (Database & Storage):** MySQL 8.x (Relational metadata engine), Redis (High-speed validation caching).

2.2 Infrastructure & Environmental Components

- . **Operating System:** Enterprise Linux (e.g., RHEL / Rocky Linux) or Windows Server.
- . **Application Server Layer:** WildFly / JBoss / Embedded Apache Tomcat (Spring Boot wrapper).
- . **Web Server & Reverse Proxy:** Nginx / Apache Web Server (SSL/TLS termination, mTLS enforcement, load balancing).
- . **Target HSM Hardware Layer:** Thales HSM (Production environment) / SoftHSM v2 (Development, isolation testing, and local simulation).
- . **Network & Environment:** Tiered LAN / WAN / VPN architecture, mandatory authoritative NTP Time Server synchronization.
- . **Log Forwarding:** Logback / Logstash / Fluentd streaming directly to a remote SIEM (Security Information and Event Management) instance.
- . **Reporting Engines:** JasperReports / OpenPDF (formatted exports), Apache Commons CSV.

3. Comprehensive Functional Requirements (FR)

3.1 CA Management, Profiles & Policies (FR-01, FR-04, FR-12)

- . **FR-01.1 Root and Sub-CA Provisioning:** The system **shall** support the initialization, hierarchical provisioning, and complete setup of Root CAs and Subordinate/Intermediate Issuing CAs using Bouncy Castle core libraries.
- . **FR-04.1 Certificate Profiles:** The system **shall** create and manage structured certificate profiles and design templates. These profiles must strictly enforce standard X.509 v3 extensions including: Basic Constraints, Key Usage, Extended Key Usage (EKU), Subject Alternative Name (SAN), CRL Distribution Points (CDP), and Authority Information Access (AIA).
- . **FR-04.2 Multi-Validity Configurations:** The system **shall** permit administrators to customize validity periods (e.g., hours, days, years) independently per profile template.
- . **FR-12.1 CP/CPS Management:** The system **shall** maintain version control and strict tracking for all system Certificate Policies (CP) and Certificate Practice Statements (CPS).

3.2 Certificate Lifecycle & Bulk Operations (FR-02, FR-10)

- . **FR-02.1 End-to-End Operations:** The system **shall** execute complete core certificate lifecycle modifications, containing: processing public requests, executing manual/automated issuances, processing renewals, managing temporary suspensions, and enacting immediate revocations.
- . **FR-02.2 Cryptographic Standard Enforcement:** All certificates minted **shall** strictly support the following algorithmic profiles:
 - . *Asymmetric Key Pairs:* RSA (2048, 3072, 4096 bit keys) and ECDSA (Curves P-256, P-384, P-521).
 - . *Hashing Mechanics:* SHA-256, SHA-384, and SHA-512.
- . **FR-02.3 Visual Certificate Viewer:** The web application **shall** provide an interactive React UI rendering hierarchical chain trees, full certificate extension maps, public metadata, and serial representations (Decimal and Hexadecimal variants).

- . **FR-10.1 Bulk Operations Engine:** The system **shall** ingest arrayed input datasets to simultaneously execute bulk certificate issuances and multi-asset revocations without degradation.

3.3 Registration Authority & API Services (FR-03, FR-11, FR-14)

- . **FR-03.1 Registration & Verification Gateway:** The system **shall** deliver a Registration Authority (RA) workspace enabling user registration, identity metadata verification, and raw PKCS#10 Certificate Signing Request (CSR) parsing.
- . **FR-03.2 Step Approval Workflows:** The system **shall** process CSR execution pipelines using automated or multi-step manual confirmation configurations before signing.
- . **FR-11.1 Secure REST API Gateway:** The system **shall** expose secure backend endpoints utilizing REST controllers. These APIs must handle machine-driven enrollment, batch issuance pipelines, real-time status tracking, and programmatic revocation commands protected by mutual TLS (mTLS).
- . **FR-14.1 Enterprise Directory Integration:** The system **shall** integrate natively with external enterprise directories (LDAP and Active Directory) and corporate Identity Providers (IdM/IAM tools) for synchronous user verification.

3.4 High-Availability Validation Services (FR-05)

- . **FR-05.1 Periodic CRL Compilation:** The system **shall** compile, digitally sign, and distribute updated Certificate Revocation Lists (CRLs), providing full support for Delta CRLs at scheduled, user-defined intervals.
- . **FR-05.2 Multi-Destination CRL Distribution:** The system **shall** publish generated binary `.crl` bundles automatically to local filesystems, public-facing web nodes via Nginx, and external directory trees (LDAP/AD).
- . **FR-05.3 Decoupled OCSP Responders:** The system **shall** feature an independent Online Certificate Status Protocol (OCSP) responder service. This service must sign validation responses using an isolated, dedicated OCSP signing certificate separate from the main CA signing key.

- **FR-05.4 High Availability & Caching:** Validation endpoints **shall** utilize Redis caching layers and Nginx load-balancing rules to maintain continuous availability under high volume.

3.5 HSM & Cryptographic Key Protections (FR-06)

- **FR-06.1 PKCS#11 Integration Layer:** The system **shall** route all root and intermediate cryptographic actions exclusively through physical Thales HSM devices or SoftHSM instances leveraging the standardized **PKCS#11** provider specification.
- **FR-06.2 Hardware Isolation Mandatory Generation:** All CA private keys **shall** be generated directly within the physical HSM perimeter boundary.
- **FR-06.3 Non-Exportable Attribute Enforcement:** Private keys residing in the HSM **shall** be flagged permanently as **non-exportable**, blocking structural extraction.
- **FR-06.4 Cryptographic Hardware Offloading:** The application **shall** offload all critical runtime digital signatures, certificate minting, data decryption, and key-wrapping sequences directly into the hardware module (conforming to FIPS 140-3 Level 2 or higher benchmarks).

3.6 Administration, RBAC & Multi-User UI (FR-08)

- **FR-08.1 Multi-Factor Authentication (MFA):** Access to management consoles **shall** require verification using passwords coupled with MFA mechanics (smart cards, digital client certificates, hardware tokens, or WebAuthn/TOTP).
- **FR-08.2 Role-Based Access Control Matrix:** Endpoints and web routes **shall** be restricted using Spring Security based on five core designated roles:
 - **CA Administrator:** Complete environmental configuration, system preferences, policy mapping, and profile initialization.
 - **RA Operator:** Individual and bulk CSR parsing, metadata vetting, manual issuance handling, and revocation processing approvals.
 - **Security Officer:** HSM slot connectivity control, token state changes, backup custody management, and security baseline validation.

- . **Auditor:** Global read-only access to system events, compliance ledgers, logs, and report export tools.
- . **End Entity:** Dedicated, self-service user workspace enabling standalone request generation and personal key/certificate downloads.
- . **FR-08.3 Unified Management Frontend:** The React user interface **shall** deliver advanced data searching, interactive chain visualizations, expiry tracking dashboards, and asynchronous drag-and-drop file upload wrappers for PEM/DER certificates and JKS/PKCS12 keystores.

3.7 Audit, Alerting, Reports & Recovery (FR-07, FR-09, FR-13)

- . **FR-07.1 Immutable Audit Logging:** The system **shall** intercept and log all operational, structural, configuration, user access, and cryptographic tasks using an append-only, tamper-proof architecture streamed out immediately via Logstash/Fluentd to a secure SIEM.
- . **FR-07.2 Signed Compliance Reporting:** Utilizing JasperReports and OpenPDF, the system **shall** generate operational audit readouts, asset expiration lists, and revocation tallies exportable to digitally signed PDF, CSV, or raw JSON.
- . **FR-09.1 Notification Routing Engine:** The system **shall** automatically dispatch notifications via Email and SMS alerting operators and end entities about imminent expiration limits, registration updates, successful issuances, and forced revocations.
- . **FR-13.1 Disaster Recovery & Automated Backups:** The system **shall** capture scheduled daily snapshots of the **MySQL transactional layer**, profile schemas, core configuration fields, and application states to guarantee compliance with targeted Recovery Time and Recovery Point Objectives (RTO/RPO). HSM private keys **shall** be backed up exclusively via secure M-of-N multi-custodian split-knowledge ceremonies.

4. Non-Functional Requirements (NFR)

- . **NFR-01 Performance & Throughput:** The system **shall** maintain capacity to process high-volume parallel cryptographic issuances and simultaneous operational validation actions without API failures.
- . **NFR-02 System Availability:** Continuous validation systems (OCSP/CRL endpoints) **shall** achieve **99.9% uptime** or higher on an annual basis.

- . **NFR-03 Horizontal Scalability:** The application tier and Nginx configurations **shall** support horizontal scaling across active nodes without data fragmentation or session dropped anomalies.
- . **NFR-04 Usability:** The user workspace **shall** provide modern design compatibility with all standard modern desktop web browsers.

5. Standard Compliance Requirements (CR)

All transactions, cryptographic outputs, network handshakes, and ledger components generated by this web application **shall** strictly comply with the following international internet engineering benchmarks:

- . **RFC 5280:** Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile.
- . **RFC 6960:** Online Certificate Status Protocol (OCSP) - Generation and Processing.
- . **RFC 3161:** Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP) for trusted validation.
- . **Regulatory Frameworks:** CA/Browser Forum Baseline Requirements, ISO/IEC 27001 Information Security Management Standard, and eIDAS compliance specifications where structurally relevant.

6. Step-by-Step Implementation Work Breakdown

PHASE 1: Core Crypto, Key Management \& Database Setup

- MySQL Schemas \& Entities
- Bouncy Castle Core Algorithms
- Keystore Manipulation Engine
- CSR \& PEM/DER/PFX Serialization

PHASE 2: CA Architecture \& Lifecycle Workflow Engines

- Root \& Intermediate CA Init
- CSR Validation \& Signing Pipeline
- Renewal \& Revocation Pipelines
- React Hierarchical Chain Tree View

PHASE 3: High-Availability Revocation Infrastructure

- Automated CRL Generation
- Directory (LDAP/AD) Publishers
- Dedicated OCSP Responder Node
- Redis Validation Caching \& Nginx

PHASE 4: PKCS#11 HSM Hardware Integration Layer

- SunPKCS11 Provider Architecture - SoftHSM v2 Container Environments
- Non-Exportable HSM Generation - Complete Cryptographic Offloading

PHASE 5: Enterprise Security Hardening, Audit \& Delivery

- Spring Security RBAC Matrices - Immutable Append-Only Logs \& SIEM
- Jasper/OpenPDF Report Engines - JUnit Automated Test \& TLS Tuning

Phase 1 – Core Crypto, Key Management & Database Setup

- . **Phase Objective:** Deploy the operational data storage backend and engineer the core cryptographic abstraction utilities.
- . **Tools & Technologies used in Phase:** Java 17+, Spring Boot, Spring Data JPA, MySQL 8.x, Maven/Gradle, Bouncy Castle Provider Core.
- . **Work Breakdown:**
 1. Deploy and initialize the MariaDB relational database matching the exact schema configurations for certificates, user profiles, templates, and logging entities.
 2. Develop core Spring Boot backend crypto helper services utilizing Bouncy Castle libraries to generate basic RSA, ECC, and EdDSA key vectors.
 3. Implement the internal Keystore Manipulation Engine enabling programmatic structural modifications inside standard formats including `.jks` and `.pfx` (PKCS#12) models.
 4. Build raw CSR generation builders to programmatically output standardized PKCS#10 strings embedded with custom user details.
 5. Construct basic binary file serializer and deserializer utilities to manage clean programmatic importing and exporting of `PEM`, `DER`, `CRT`, and `PFX` extensions.

Phase 2 – CA Architecture & Certificate Lifecycle Workflow Engines

- . **Phase Objective:** Formulate the internal hierarchical Certificate Authority architecture and finalize end-entity certificate lifecycles.
- . **Tools & Technologies used in Phase:** Spring Boot Framework Core, React, TailwindCSS, Axios API Client, Spring Boot Starter Mail.

- **Work Breakdown:**

1. Build administrative configurations allowing an operator to instantiate a self-signed Root CA context profile.
2. Develop explicit execution paths allowing the Root CA to securely sign and activate downstream Intermediate/Subordinate CAs.
3. Program the primary CSR Signing Engine to take an authenticated PKCS#10 incoming byte block and sign it using the designated Sub-CA key via Bouncy Castle.
4. Build lifecycle route logic and matching state changes to handle standard renewal actions, temporary certificate suspensions, and formal revocation flags.
5. Develop the front-end interactive Certificate Viewer components using React to visually represent trust chains, certificate extensions, and serial records.

Phase 3 – High-Availability Revocation Infrastructure

- **Phase Objective:** Stand up the critical external interfaces used by verification applications to confirm validity and revocation states in real time.
- **Tools & Technologies used in Phase:** Spring MVC, Redis, Nginx Load Balancer, Apache Directory API / UnboundID LDAP SDK.
- **Work Breakdown:**

1. Develop background cron operations inside Spring Boot using `@Scheduled` parameters to periodically build and sign the system's latest Certificate Revocation List (CRL).
2. Construct publishing micro-tasks to transmit compiled `.crl` files to local storage destinations and external corporate LDAP/AD endpoints.
3. Build the high-speed, standalone OCSP web responder endpoint designed to parse binary incoming payload arrays and output cryptographically signed validation responses.
4. Integrate Redis into the validation service to cache state checks, reducing the query overhead on the underlying MySQL layer.
5. Configure Nginx configurations with active upstream balancing definitions to split inbound validation traffic securely across multiple horizontal app server clones.

Phase 4 – PKCS#11 HSM Hardware Integration Layer

- . **Phase Objective:** Migrate the application root of trust from software key containers to hardware-isolated boundary protections.
- . **Tools & Technologies used in Phase:** Java SunPKCS11 Provider, SoftHSM v2 (Docker/Local testing packages), Thales HSM Driver Tooling.
- . **Work Breakdown:**
 1. Develop structural configuration profiles mapping Java's internal SunPKCS11 provider module cleanly to HSM slot pointers.
 2. Build local validation environments using SoftHSM v2 container instances to verify PIN structures, operational slots, and target token initializations during code verification.
 3. Update key generation logic so that private keys are created directly within the HSM hardware boundary and permanently assigned non-exportable properties.
 4. Refactor backend service classes so that runtime digital signatures and certificate processing offload cryptographic calculation vectors to the connected HSM.

Phase 5 – Enterprise Security Hardening, Audit & Delivery

- . **Phase Objective:** Lock down system interfaces, integrate complete auditing reporting features, and provision production-ready configurations.
- . **Tools & Technologies used in Phase:** Spring Security Core, Nimbus JOSE-JWT, Logback/Logstash Framework, JasperReports / OpenPDF, JUnit 5, SonarQube, Nginx SSL Configurations.
- . **Work Breakdown:**
 1. Secure all system endpoints using Spring Security filters, mandating multi-factor verification checks and validating stateless incoming JWT claims.
 2. Build and connect comprehensive aspect interceptors to log structural key usage, user logins, and administrative changes to an append-only ledger, feeding Logstash directly.
 3. Enforce advanced database cell encryption or configuration wrapper encryption for critical passwords and at-rest configurations.
 4. Build analytical export modules using JasperReports and OpenPDF to deliver cleanly formatted, downloadable system status sheets in PDF, CSV, and JSON.

5. Author modular JUnit automated test frameworks, conduct static code analysis using SonarQube to verify code cleanliness, tune operational database connection thread pools, and lock down Nginx with strict TLS 1.3 mTLS configurations.